PROYEK AKHIR · INTEGRASI SISTEM DAN KOMPUTASI AWAN

Kasir terhubung, stok terkendali

Analisis arsitektur integrasi untuk ekosistem kasir digital dan otomasi rantai pasok Warung Kelontong Pak Hendy.

Mitra: Warung Kelontong Pak Hendy

Kelompok 67: Difki Junaedi · Nurul Hidayah · Bima Azmi Khoerullah

Kurikulum: STMIK Bandung, 2025

RINGKASAN EKSEKUTIF

Warung Pak Hendy memerlukan alur data yang menyatukan transaksi kasir, saldo persediaan, dan pemesanan ulang kepada agen grosir. Ketiganya memiliki kebutuhan waktu dan tingkat keandalan yang berbeda. Karena itu, rancangan ini memilih REST API sinkron untuk konfirmasi penjualan dan pengurangan stok, lalu menggunakan RabbitMQ untuk menyampaikan peristiwa ambang stok kepada layanan pemesanan pemasok secara asinkron.

Inventory Backend berbasis Laravel dan MySQL menjadi *single source of truth* untuk katalog, saldo stok, serta mutasi. Pengurangan stok dijalankan dalam transaksi ACID dengan penguncian baris. Pola transactional outbox menghubungkan komit basis data dengan publikasi pesan. Consumer pemasok memproses pesan secara idempoten menggunakan `event_id` sehingga pengiriman ulang tidak menghasilkan pesanan ganda.

> **Batasan data:** dokumen kasus tidak menyertakan angka transaksi harian, luas operasi, nilai ambang stok, SLA agen, maupun hasil wawancara. Karena itu, profil dan rancangan kuantitatif di bawah bersifat kualitatif atau usulan awal. Semua parameter operasional harus divalidasi bersama Pak Hendy sebelum implementasi.

Bab 1 · Konteks mitra dan proses bisnis saat ini

1.1 Profil organisasi dan skala usaha

Warung Kelontong Pak Hendy merupakan usaha ritel kebutuhan harian yang melayani pembelian satuan maupun kebutuhan rumah tangga dalam jumlah lebih besar. Portofolio barang mencakup produk dengan karakter perputaran berbeda: minyak goreng, beras, gula, dan mi instan termasuk kelompok yang perlu dipantau rutin karena permintaan cenderung berulang. Namun, frekuensi penjualan aktual dan jumlah unit per hari belum tersedia. Sebelum sistem menetapkan kapasitas, ambang pemesanan, atau target layanan, tim perlu mengukur transaksi per jam, volume penjualan per SKU, variasi permintaan akhir pekan, dan waktu pemenuhan pemasok.

1.2 Proses bisnis eksisting (As-Is)

1. **Transaksi kasir.** Kasir menerima barang, menghitung harga menggunakan daftar harga atau ingatan/alat bantu yang tersedia, lalu menuliskan nota kertas atau mencatat penjualan secara manual. Total pembayaran dan rincian barang dapat tersimpan pada nota, tetapi belum tentu menjadi catatan terstruktur yang otomatis mengubah saldo stok.
2. **Pengecekan stok.** Pemilik atau petugas memeriksa barang di rak dan gudang secara fisik. Catatan stok administratif, jika ada, diperbarui terpisah dari transaksi. Selisih dapat muncul akibat barang terjual, rusak, dipakai internal, atau salah hitung yang tidak dicatat sebagai mutasi.
3. **Pemesanan ulang.** Saat stok dianggap menipis, pemilik menghubungi satu atau lebih agen grosir melalui WhatsApp. Daftar dan jumlah pesanan disusun manual. Status ketersediaan, konfirmasi harga, estimasi kedatangan, dan penerimaan barang perlu ditelusuri dari percakapan.

1.3 Pihak yang terlibat

Pihak	Tanggung jawab saat ini	Kebutuhan sistem
Kasir	Mencatat barang dan pembayaran pada transaksi pelanggan.	Alur input ringkas, harga jelas, hasil transaksi langsung diketahui, dan perlindungan dari saldo stok tidak cukup.
Pengelola/Pemilik (Pak Hendy)	Mengawasi stok, menentukan barang yang perlu dibeli, merekonsiliasi penerimaan dan penjualan.	Visibilitas stok dan riwayat mutasi, daftar rekomendasi restock, serta status pesanan pemasok.
Agen supplier grosir	Menerima pesanan, mengonfirmasi ketersediaan/harga, dan memenuhi barang.	Pesanan terstruktur dan dapat ditelusuri; integrasi otomatis bergantung pada kanal/kemampuan teknis yang disepakati.

Bab 2 · Persoalan integrasi dan data silo

2.1 Pemetaan silo

Domain data	Lokasi/cara simpan saat ini	Masalah integrasi
Penjualan dan pembayaran	Nota kertas atau pencatatan kasir.	Rincian transaksi tidak selalu tersedia bagi pencatatan stok dan laporan penutupan hari.
Saldo barang	Kondisi fisik rak/gudang dan catatan terpisah.	Saldo administratif tertinggal dari realitas setelah transaksi, barang rusak, atau penerimaan.
Pesanan pemasok	Chat WhatsApp dan ingatan pemilik.	Format, status, harga, dan waktu pemenuhan tidak tersentralisasi serta sulit diaudit.
Data produk	Daftar harga, label barang, atau pengetahuan petugas.	Nama, satuan, harga, dan identitas SKU dapat berbeda antarcatatan.

Inti persoalan bukan hanya penggunaan media berbeda, melainkan ketiadaan pemilik data dan mekanisme sinkronisasi yang tegas. Jika penjualan terjadi tanpa mutasi stok tercatat, saldo sistem tidak dapat menjadi dasar keputusan pembelian. Sebaliknya, jika stok hanya diperbarui saat hitung fisik berkala, keputusan restock selalu menggunakan keadaan yang sudah usang.

2.2 Konsekuensi bisnis

- **Overselling:** kasir dapat menerima penjualan atas barang yang sesungguhnya sudah habis. Dampaknya berupa pembatalan, penggantian barang, pengalaman pelanggan yang buruk, dan koreksi stok manual.
- **Lost sales opportunity:** stok fast-moving yang terlambat dipesan membuat permintaan tidak terpenuhi selama lead time pengadaan. Sistem perlu mempertimbangkan stok tersedia, stok yang sudah dipesan, konsumsi rata-rata, dan waktu pasok, bukan sekadar angka stok fisik.
- **Rekonsiliasi akhir hari tidak akurat:** nota yang terlewat, salah hitung, retur, diskon, atau pembayaran yang tidak dikaitkan dengan transaksi menyulitkan pencocokan kas dengan total penjualan.
- **Jejak audit terbatas:** perubahan stok tanpa alasan, pelaku, dan waktu yang tercatat sulit ditelusuri ketika terjadi selisih.

> **Implikasi arsitektural:** setiap transaksi yang berhasil harus menghasilkan perubahan stok dan catatan penjualan secara atomik. Pesan pemesanan pemasok tidak perlu menahan kasir; kegagalan kanal pemasok harus dikelola pada alur asinkron dengan status yang dapat dipantau.

Bab 3 · Evaluasi gaya integrasi

Evaluasi berikut menggunakan empat gaya integrasi Enterprise Integration Patterns. Pemilihan tidak bersifat seragam untuk seluruh sistem. Jalur yang memerlukan keputusan langsung menggunakan request-response; proses yang dapat tertunda menggunakan pesan asinkron.

3.1 File Transfer

Ekspor dan impor CSV mudah diterapkan, dapat dibuka dengan perangkat umum, serta cocok untuk migrasi awal atau pertukaran batch dengan agen yang belum memiliki API. Namun, file tidak memberi konfirmasi transaksi seketika, rawan versi kolom dan duplikasi impor, serta membutuhkan prosedur rekonsiliasi. File transfer ditolak untuk transaksi kasir realtime karena kasir perlu mengetahui pada saat itu juga apakah penjualan dan pengurangan stok berhasil.

3.2 Shared Database

Satu basis data bersama tampak sederhana dan memungkinkan komponen membaca data terbaru tanpa mekanisme sinkronisasi terpisah. Konsekuensinya, layanan menjadi terikat pada skema, indeks, hak akses, dan aturan transaksi yang sama. Perubahan skema satu komponen dapat merusak komponen lain; beban konkurensi dapat memicu lock contention; dan batas kepemilikan data menjadi kabur. Pendekatan ini tidak dipilih untuk komunikasi antar-layanan independen. Dalam rancangan ini, hanya Inventory Backend yang memiliki basis data operasionalnya. Outbox adalah tabel internal layanan tersebut, bukan basis data bersama.

3.3 Remote Procedure Invocation: REST sinkron

POS memanggil `POST /api/v1/transaksi` pada Inventory Backend. Backend memvalidasi baris barang, harga, dan ketersediaan; menulis transaksi serta mutasi stok; lalu mengembalikan hasil. Respons `201 Created` menyatakan transaksi berhasil dibuat. Respons `409 Conflict` menyatakan stok tidak cukup atau terjadi konflik bisnis dan transaksi tidak dikomit.

Keunggulan utamanya ialah konfirmasi deterministik sebelum kasir menutup transaksi. Trade-off-nya adalah *temporal coupling*: kedua layanan harus dapat berkomunikasi pada saat permintaan dilakukan. Latensi API juga masuk ke waktu tunggu kasir. Sistem perlu timeout terukur, koneksi yang aman, penanganan retry, serta kunci idempotensi transaksi agar timeout setelah komit tidak menyebabkan penjualan tercatat dua kali. Antarmuka kasir sebaiknya menampilkan status gagal/tertunda secara jelas, bukan menganggap timeout sebagai transaksi sukses.

3.4 Asynchronous Messaging: RabbitMQ

Setelah stok turun melewati kebijakan ambang, Inventory Backend membuat event dan menaruhnya dalam outbox pada transaksi yang sama. Dispatcher memublikasikan event ke topic exchange RabbitMQ. Supplier Order Gateway mengonsumsi event dan membentuk usulan atau pesanan pemasok sesuai aturan yang telah disetujui pemilik.

Pesan asinkron mengurangi kopling temporal antara layanan inventaris dan gateway. Jika gateway atau kanal eksternal sedang tidak tersedia, transaksi kasir tetap dapat selesai; pesan menunggu untuk diproses kemudian. Keandalan membutuhkan durable exchange/queue, pesan persistent, publisher confirms, acknowledgement setelah pemrosesan berhasil, retry terbatas, dan dead-letter queue untuk pesan yang terus gagal. Pengiriman dapat terjadi setidaknya sekali, sehingga consumer wajib idempoten. Status pemesanan bersifat *eventually consistent*: perubahan stok dan status pesanan tidak selalu tampak serentak.

3.5 Matriks perbandingan

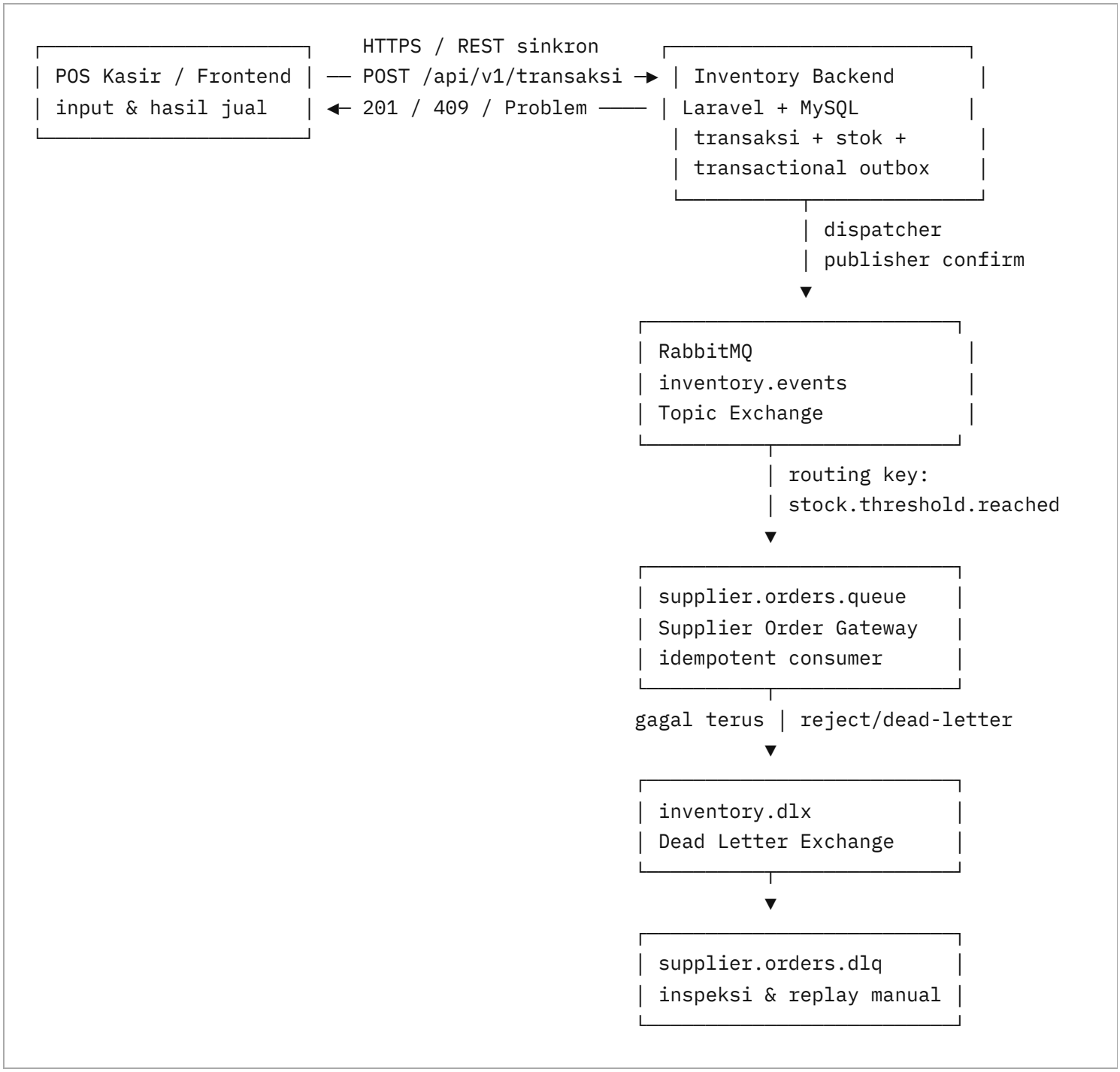
Gaya	Coupling	Keandalan	Kompleksitas	Latensi	Use case
File Transfer	Rendah saat pertukaran, tinggi pada format/operasi batch.	Perlu validasi, arsip, dan rekonsiliasi; tidak realtime.	Rendah–sedang.	Tertunda per batch.	Migrasi awal atau laporan pemasok.
Shared Database	Tinggi; skema dan transaksi dipakai bersama.	ACID terpusat, tetapi gangguan/lock berdampak luas.	Rendah di awal, tinggi ketika layanan berkembang.	Rendah untuk baca lokal.	Tidak dipilih antar-layanan independen.
REST sinkron	Temporal coupling dan kontrak API.	Respons langsung; memerlukan timeout dan idempotensi.	Sedang.	Request-response.	POS → Inventory: validasi serta konfirmasi penjualan.
RabbitMQ asinkron	Longgar; produsen dan konsumen terpisah.	Durable delivery, confirms, ACK, retry, dan DLQ.	Sedang–tinggi.	Asinkron; eventual consistency.	Inventory → Supplier Gateway: restock dan pemrosesan pemasok.

Bab 4 · Perancangan arsitektur dan integrasi

4.1 Topologi layanan

- **POS Kasir / Frontend:** UI pencatatan penjualan. Mengirim permintaan REST, menampilkan hasil transaksi, dan menyimpan kunci idempotensi untuk retry aman.
- **Inventory Backend Service (Laravel + MySQL):** otoritas katalog, harga, saldo, transaksi, mutasi, kebijakan ambang, dan outbox.
- **Supplier Order Gateway / Consumer:** worker yang menerima event, mencegah pemrosesan ganda, menerapkan kebijakan pemesanan, serta mengirim pesanan lewat kanal yang benar-benar tersedia. Integrasi WhatsApp otomatis tidak diasumsikan; opsi awal dapat berupa daftar pesanan yang perlu disetujui pemilik.
- **RabbitMQ:** broker untuk event inventaris, antrian pesanan, retry terkontrol, dan dead lettering.

4.2 Diagram arsitektur



Gambar 1. Jalur penjualan sinkron terpisah dari jalur pemesanan pemasok asinkron dan dead-letter.

Konfigurasi broker perlu mendeklarasikan `inventory.events` sebagai topic exchange durable, binding dari routing key yang disepakati ke `supplier.orders.queue`, serta dead-letter exchange `inventory.dlx` yang mengarahkan pesan gagal ke `supplier.orders.dlq`. DLQ harus dipantau dan memiliki prosedur inspeksi, koreksi, lalu replay terkontrol; DLQ bukan tempat pembuangan yang diabaikan.

4.3 Ketahanan dan integritas data

Penguncian pesimistis dan transaksi ACID

Untuk setiap SKU yang dijual, backend memulai transaksi basis data, mengunci baris stok memakai `lockForUpdate()`, memvalidasi bahwa saldo tersedia mencukupi, lalu mencatat transaksi dan mutasi pengurangan. Jika semua validasi berhasil, transaksi dikomit; jika ada kegagalan, seluruh perubahan di-rollback. Penguncian mencegah dua kasir membaca saldo yang sama lalu menjual unit terakhir secara bersamaan. Baris SKU sebaiknya dikunci dalam urutan ID konsisten untuk mengurangi risiko deadlock. Pesan event dan operasi jaringan eksternal tidak boleh dilakukan sambil memegang lock stok.

Transactional Outbox

Menulis stok ke MySQL lalu langsung menerbitkan pesan ke RabbitMQ memiliki celah: database dapat berhasil commit, sementara broker sedang down. Outbox menyimpan event dalam transaksi database yang sama dengan mutasi stok. Proses dispatcher mengambil baris yang belum terkirim, menerbitkan pesan persistent, menunggu publisher confirm, lalu menandai baris terkirim. Jika proses jatuh setelah publish tetapi sebelum penandaan, pesan mungkin dipublikasikan ulang; desain karena itu tetap memerlukan idempotensi di consumer. Outbox menghindari kehilangan event, tetapi bukan klaim bahwa event hanya terkirim sekali.

Idempotent Consumer

Setiap event memiliki UUID `event_id`. Gateway menyimpan ID yang telah diproses dalam tabel inbox/processed-events dengan batas unik. Pencatatan ID dan pembuatan permintaan pesanan internal dilakukan atomik. Jika event yang sama datang kembali, consumer mendeteksi duplikasi dan meng-ACK tanpa membuat pesanan kedua. Untuk efek eksternal yang kanalnya tidak mendukung idempotency key, gateway perlu menyimpan status pengiriman dan melakukan rekonsiliasi sebelum mencoba ulang agar retry tidak menggandakan pesanan.

Aturan stok dan pemesanan ulang

Ambang pemesanan sebaiknya mempertimbangkan lead time dan permintaan, misalnya titik pemesanan ulang berdasarkan konsumsi selama lead time ditambah safety stock. Angka tersebut harus dihitung dari data penjualan aktual dan variasi pemasok. Event pemicu tidak otomatis berarti pesanan harus langsung dikirim: gateway dapat membuat draft yang disetujui pemilik, terutama pada tahap awal, sampai kebijakan jumlah minimum, substitusi barang, harga, dan pemasok tervalidasi.

Bab 5 · Kontrak API, event, dan roadmap checkpoint 1

5.1 Ringkasan kontrak API (RFC 9457)

Endpoint transaksi menggunakan `POST /api/v1/transaksi` melalui HTTPS. Request memuat identitas idempotensi pada header `Idempotency-Key`, identitas kasir, waktu transaksi, metode pembayaran, serta item berisi SKU dan kuantitas. Harga dan nama produk otoritatif ditentukan backend; POS tidak boleh menjadi sumber kebenaran harga.

Status	Makna	Tindakan POS
`201 Created`	Transaksi tersimpan, saldo stok telah dimutasi.	Tampilkan nomor transaksi dan ringkasan pembayaran.
`400 Bad Request`	Struktur request tidak valid.	Perbaiki input; jangan retry request identik tanpa perubahan.
`404 Not Found`	SKU tidak dikenal atau tidak aktif.	Minta kasir memilih barang katalog yang valid.
`409 Conflict`	Stok tidak cukup, kunci idempotensi digunakan dengan payload berbeda, atau konflik domain.	Jangan tandai transaksi sukses; muat ulang saldo atau tinjau transaksi.
`422 Unprocessable Content`	Validasi semantik gagal, misalnya kuantitas tidak positif.	Tampilkan rincian field yang perlu diperbaiki.
`503 Service Unavailable`	Backend sementara tidak dapat melayani.	Berikan status belum terkonfirmasi dan retry aman dengan kunci yang sama.

Galat menggunakan `Content-Type: application/problem+json` sesuai RFC 9457. Contoh konflik stok:

```
{
  "type": "https://warung.example/problems/insufficient-stock",
  "title": "Stok tidak mencukupi",
  "status": 409,
  "detail": "Kuantitas yang diminta melebihi stok tersedia.",
  "instance": "/api/v1/transaksi",
  "code": "INSUFFICIENT_STOCK",
  "items": [
    { "sku": "SKU-beras-5kg", "requested": 2, "available": 1 }
  ],
  "trace_id": "01J8Q2M6R4Y7K9P3T5V1W0XABC"
}
```

Field inti `type`, `title`, `status`, `detail`, dan `instance` mengikuti Problem Details. Ekstensi seperti `code`, `items`, dan `trace_id` mendukung penanganan UI serta pelacakan. Informasi internal, query SQL, kredensial, dan stack trace tidak boleh dikirim ke klien.

5.2 Spesifikasi event asinkron

Nama event: `StockThresholdReached`. Versi skema dicantumkan agar perubahan kompatibel dapat dikelola. Nilai threshold dan stok adalah snapshot saat event diterbitkan; gateway harus memeriksa kondisi terkini sebelum menyusun pesanan bila event telah tertunda.

```
{
  "event_id": "8cb8840a-2159-4b8f-bc67-e39be9df9bc0",
  "event_type": "StockThresholdReached",
  "schema_version": "1.0",
  "occurred_at": "2026-10-20T08:15:30Z",
  "producer": "inventory-backend",
  "data": {
    "sku": "SKU-minyak-1L",
    "product_name": "Minyak goreng 1 L",
    "available_quantity": 8,
    "reorder_threshold": 10,
    "unit": "botol",
    "location_id": "TOKO-UTAMA"
  },
  "correlation_id": "trx-01J..."
}
```

Routing key yang disepakati: `stock.threshold.reached`. Pesan tidak memuat rahasia atau data pelanggan. Jumlah rekomendasi pembelian dapat ditambahkan pada versi skema berikutnya setelah aturan stok minimum dan lead time disepakati.

5.3 Roadmap dan tanggung jawab Kelompok 67

Roadmap checkpoint 1 berikut adalah usulan urutan kerja; tanggal kalender harus disesuaikan dengan jadwal dosen dan mitra. Ketiga anggota meninjau kontrak integrasi bersama agar keputusan antarmuka tidak terpecah.

- **Tahap 1 · Validasi proses dan data**
  Difki Junaedi memimpin wawancara proses kasir dan pemetaan integrasi; Nurul Hidayah mengumpulkan sampel katalog/stok; Bima Azmi Khoerullah mendokumentasikan pemasok dan alur pemesanan. Keluaran: As-Is tervalidasi, daftar SKU contoh, transaksi harian dan variasi permintaan yang diukur, serta daftar asumsi yang tersisa.

- **Tahap 2 · Kontrak dan rancangan**
  Kelompok menyepakati payload transaksi, status API, Problem Details, skema event, routing key, dan kebijakan retry. Keluaran: diagram komponen final, definisi data, threat/risiko operasional dasar, serta kriteria penerimaan checkpoint.

- **Tahap 3 · Prototipe integrasi**
  Implementasikan jalur REST sampai transaksi mengurangi stok secara atomik; lanjutkan outbox, RabbitMQ, consumer idempoten, dan DLQ. Jalankan uji konkurensi stok, replay event duplikat, serta simulasi broker tidak tersedia.

- **Tahap 4 · Demo dan evaluasi checkpoint 1**
  Demonstrasikan penjualan berhasil (`201`), stok tidak cukup (`409`), event ambang stok yang diproses satu kali, dan pesan gagal yang masuk DLQ. Catat hasil, batasan, keputusan mitra, serta pekerjaan lanjutan.

Anggota	Fokus koordinasi	Keluaran utama
Difki Junaedi	Proses kasir, API transaksi, dan sinkronisasi kebutuhan UI.	Alur transaksi, kontrak request/response, skenario validasi.
Nurul Hidayah	Katalog, saldo, mutasi, aturan ambang, dan integritas data.	Model data inventaris, aturan stok, pengujian ACID/konkurensi.
Bima Azmi Khoerullah	RabbitMQ, gateway pemasok, retry, idempotensi, dan DLQ.	Skema event, konfigurasi alur pesan, prosedur pemulihan.

Pembagian di atas membagi fokus, bukan membatasi kepemilikan. Setiap anggota perlu melakukan review silang terhadap kontrak dan skenario uji.

Kesimpulan

Arsitektur yang diusulkan memisahkan konfirmasi penjualan dari otomasi restock sesuai kebutuhan waktunya. POS menggunakan REST sinkron agar transaksi memperoleh keputusan langsung dari sumber stok yang otoritatif. Inventory Backend mengelola komit transaksi dan outbox secara atomik, sementara RabbitMQ menyampaikan event ke Supplier Order Gateway tanpa membuat kasir bergantung pada ketersediaan kanal pemasok. Penguncian pesimistis, idempotensi, publisher confirms, retry, dan DLQ membentuk lapisan perlindungan terhadap race condition, kehilangan pesan, serta pengiriman ulang.

Sebelum implementasi produksi, Kelompok 67 perlu mengukur pola penjualan nyata, mengonfirmasi alur pembayaran dan retur, menentukan ambang stok per SKU, serta menyepakati apakah pesanan pemasok otomatis dikirim atau terlebih dahulu memerlukan persetujuan Pak Hendy. Keputusan tersebut mengubah kebijakan bisnis, bukan sekadar konfigurasi teknis.

Catatan akademik: analisis ini merupakan rancangan konseptual berdasarkan studi kasus yang diberikan. Nilai operasional yang belum tersedia tidak direkayasa dan perlu dikonfirmasi saat pengumpulan kebutuhan.

Kelompok 67 · STMIK Bandung · Integrasi Sistem dan Komputasi Awan · Kurikulum 2025